جامعة المجمعة
Majmaah University
College of Computer and Information Sciences

CS231 Data Structures

Team Project Report

Airline Management System

Team Members	*1. Mohammed Al-badiah , 461100923*
	2. Othman Al-Thabit , 462104364
	3. Rakan Al-harbi , 461102405
	4. Abdullah Al-nooh , 461105858
	5. Ahmed Al-otaibi , 451102854
Section	*: 224*
Instructor	*Hamad Alabduljabbar*
Semester	*Spring 2026*
Date	*9/5/2026*
GitHub Repo	*https://github.com/mhammdalbadiah/AirlineSystem.git*

Table of Contents

1	Introduction
2	Class Design
3	Part 1: Linked List Implementation
4	Part 2: Stack Implementation
5	Part 3: Queue Implementation
6	Challenges and Lessons Learned
7	Team Member Contributions
8	Conclusion

1. Introduction

The Airline Management System is a CPP console App we built for our CS231 DS course The system is designed to handle basic airline operations by managing four main entities : Passengers , Flights , Booking Offices , and Tickets Through a simple menu interface , the user can add , view , search , update , and remove any of these entities

The system works by having a Booking Office issue a Ticket that links a Passenger to a Flight. Every Flight keeps track of all the Tickets issued for it , so when you open a flight's profile you can see every booking connected to it. This is meant to reflect how a real airline operates the office handles the booking side and the flight tracks who is on it

The main goal of this project was to practice using data structures in a real and meaningful way. We implemented the whole system three times using three different structures , a Linked List , a Stack , and a Queue , Each structure manages the same four data types but in a completely different way , which helped us understand the strengths and limitations of each one

2. Class Design

The system is built on four classes , each representing one of the main entities. Every class is in its own header file with private data members and setter functions that validate input before saving anything

2.1 BookingOffice Class

The Booking Office class represents a ticket office in the airline. It stores three data members : Office ID , Office Name , and Office Location. The ID must be exactly 10 digits , while the name and location must be between 3 and 20 characters. In the system , this class plays the role of the entity that creates and issues tickets to passengers for their flights

2.2 Passenger Class

The Passenger class stores the basic information of a traveler. Its three data members are ID , Name , and Passport Number. The ID must be exactly 10 digits , the name between 3 and 20 characters , and the passport number between 5 and 15 alphanumeric characters. A passenger does not exist independently in the system , they are always part of a ticket that connects them to a specific flight

2.3 Ticket Class

The Ticket class is the link between a passenger , a flight , and an office. It stores Ticket Number , Flight Number , Office Name , and a Passenger List. When an office books a passenger onto a flight , a ticket is created to record that transaction. The passenger list inside a ticket cannot be empty , at least one passenger must be attached to every ticket

2.4 Flight Class

The Flight class represents a scheduled flight. It stores Flight ID , Destination , Gate , Departure Time , and a Ticket List. What makes this class different from the others is the ticket list , every ticket issued for this flight gets stored inside it. When a flight profile is displayed , all its associated tickets are shown , giving a full picture of everyone booked on that flight and through which office

3. Part 1: Linked List Implementation

3.1 Explanation

A Linked List is a chain of nodes where each node holds a data item and a pointer to the next node in the list. There is no fixed size , nodes are created dynamically in memory as needed and linked together through pointers

We implemented a generic Linked List template so the same code works for all four data types without any duplication. The list tracks the first node through a head pointer and keeps a size counter for how many nodes exist

Insert walks to the last node and attaches the new one at the end. Delete walks to the node just before the target position , rewires its pointer to skip the deleted node , then frees the memory , the order of these steps is critical. Modify uses a helper function to reach the target position and replaces its data with new input , Find walks from the head and compares each node's key field to the search value , Display simply walks from head to end and prints every node

3.2 Key Code Snippets

```
Node<T>* getNodeAt(int pos) {

        if  pos < 1 || pos > size)
        return nullptr
                        ;
        Node<T>* current = head
                                ;
        for  int i = 1 ; i < pos ; i++)
         current = current->next ;

        return current
    }                   ;
```

```
void deleteByPosition(int pos) {

        // Check if list is empty
        if  head == nullptr) {
          (
            cout << "List is empty , nothing to delete !! " << endl;
            return
        }         ;

        // Check if position is valid
        if  pos < 1 || pos > size) {
          (
            cout << "Invalid position Must be between 1 and "<<size<< endl;
            return
        }         ;

        Node<T>* temp
                    ;
        // Special case : deleting the first node
        if  pos == 1) {
          (
            temp = head
            head = head;->next
            delete temp      ;
                        ;
        } else
              {
            // Traverse to the node just before the target position
            Node<T>* current = head
                                    ;
            for  int i = 1 ; i < pos - 1  ; i++)
             current = current->next ;

            temp = current->next              // Node to be deleted
            current->next = temp;->next       // Skip over the deleted node
            delete temp                ;
        }              ;

        size--
        cout <; "Deleted successfully ! " << endl;
    }
```

```
void insert(T data) {

        Node<T>* newNode = new Node<T>(data) ;  // Create a new node

         if  head == nullptr)
           (head = newNode
            else         ;
                {
           // Traverse to the last node
           Node<T>* current = head
                                  ;
           while  current->next != nullptr) {
               current = current->next
           }                          ;

           // Attach new node at the end
           current->next = newNode
        }                         ;

        size++
        cout << "Inserted successfully !  " << endl;
    }
```

3.3 Screenshots

```
==========================================
          Passenger Linked List
==========================================
1) Display All Passengers
2) Insert Passenger
3) Delete Passenger
4) Modify Passenger
5) Find Passenger
0) Back to Main Menu
==========================================
Enter your choice  : 2

##################################
Please Enter Passenger ID : 0000000001
##################################

##################################
Please Enter Passenger Name : Mohammed Al-badiah
##################################

##################################
Please Enter Passport Number : 001
Passport Number length must be between 5 and 15 !
##################################
Please Enter Passport Number : 00001
##################################

Inserted successfully !

==========================================
```

```
==========================================
          Passenger Linked List
==========================================
1) Display All Passengers
2) Insert Passenger
3) Delete Passenger
4) Modify Passenger
5) Find Passenger
0) Back to Main Menu
==========================================
Enter your choice  : 1

--- Passenger #1 ---
=====================================
          Passenger Profile
=====================================
ID           : 0000000001
Name         : Mohammed Al-badiah
Passport No  : 00001

--- Passenger #2 ---
=====================================
          Passenger Profile
=====================================
ID           : 0000000002
Name         : Othman Al-Thabit
Passport No  : 00002
```

```
=========================================
          Passenger Linked List
=========================================
1) Display All Passengers
2) Insert Passenger
3) Delete Passenger
4) Modify Passenger
5) Find Passenger
0) Back to Main Menu
=========================================
Enter your choice  : 3

Enter position to delete (1 - 2) : 2
Deleted successfully !
```

4. Part 2: Stack Implementation

4.1 Explanation

A Stack is a data structure that follows LIFO Last In , First Out , The last item you add is always the first one to come out , like a stack of plates in a cafeteria , You can only interact with the top there is no way to reach something in the middle or at the bottom without removing everything above it first

We built the Stack using the same Node structure from Part 1 , but instead of a head pointer we use a top pointer that always points to the most recently added item. This is the key difference from the Linked List , you lose positional access completely , but the structure becomes simpler and faster for its specific use case

Push replaces Insert , a new node is created , its next pointer is set to the current top , then it becomes the new top. Pop replaces Delete , the top node is saved , the top pointer drops down one level , then the saved node is freed from memory. Peek is a new operation that simply returns the top node without removing it , useful when you just want to see what is there. Display still works the same way , it walks from top to bottom and prints every item

4.2 Key Code Snippets

```
void pop() {

        if  top == nullptr) {
          ( cout << "Stack is empty nothing to pop ! " << endl;
            return
        }         ;

        Node<T>* temp = top ;
        top = top->next ;
        delete temp ;

        size--
        cout << "Popped succesfully !" << endl;
    }
```

```
void push(T data) {

        Node<T>* newNode = new Node<T>(data) ;
        // Create new node

        newNode->next = top ;
        // New node points to the old top
        top = newNode ;
        // New node becomes the new top

        size++
        cout << "Pushed successfully !!" << endl;

}
```

```
Node<T>* peek() { return top ;}
```

Diplay From “ Menus.cpp “

```
Node<T>* current = pStack.getTop() ;
int counter = 1
               ;
while (current != nullptr) {

    cout << "--- #" << counter << " (from top) -- " << endl;
    current->data.displayPassenger() ;
    current = current->next ;
    counter++
}            ;
```

4.3 Screenshots

```
==========================================
                Flight Stack
==========================================
1) Display All (Top to Bottom)
2) Push Flight
3) Pop Flight
4) Peek (View Top)
5) Find Flight
0) Back to Main Menu
==========================================
Enter your choice : 3

Popped succesfully !
```

```
==========================================
                Flight Stack
==========================================
1) Display All (Top to Bottom)
2) Push Flight
3) Pop Flight
4) Peek (View Top)
5) Find Flight
0) Back to Main Menu
==========================================
Enter your choice : 2

##################################
Please Enter Flight ID : 0
Invalid input , length must be between 2 and 25 !!
##################################
Please Enter Flight ID : 01
##################################

##################################
Please Enter Destination : RUH
##################################

##################################
Please Enter Gate : A1
##################################

##################################
Please Enter Departure Time : 14:00
##################################

Pushed successfully !!
```

```
=========================================
                Flight Stack
=========================================
1) Display All (Top to Bottom)
2) Push Flight
3) Pop Flight
4) Peek (View Top)
5) Find Flight
0) Back to Main Menu
=========================================
Enter your choice : 5

################################
Enter Flight ID to search  003
Found at Level #1 from top :
=====================================
            Flight Profile
=====================================
Flight ID       : 003
Destination     : JED
Gate            : Q22
Departure Time  : 18:45
Issued Tickets  :
No Tickets Found !
```

```
=========================================
                Flight Stack
=========================================
1) Display All (Top to Bottom)
2) Push Flight
3) Pop Flight
4) Peek (View Top)
5) Find Flight
0) Back to Main Menu
=========================================
Enter your choice : 4

Top of Stack :
=====================================
            Flight Profile
=====================================
Flight ID       : 003
Destination     : JED
Gate            : Q22
Departure Time  : 18:45
Issued Tickets  :
No Tickets Found !
```

5. Part 3: Queue Implementation

5.1 Explanation

A Queue is a data structure that follows FIFO , First In , First Out. The first item you add is the first one to come out , exactly like a real waiting line at an airport. New items always join at the back and leave from the front nobody can skip the line

We built the Queue using two pointers : front which points to the first item waiting to leave , and back which points to the last item that just joined , Having both pointers means we always know exactly where to add and where to remove without having to traverse the whole list every time

Enqueue replaces Insert , the new node is attached after the back pointer and then back moves to point at it. If the queue was empty , both front and back point to the new node since it is the only one there. Dequeue replaces Delete , the front node is saved , front moves to the next node , and the saved node is freed from memory , If the queue becomes empty after dequeue , back is also reset to null so the queue stays in a clean state. Peek returns the front node without removing it , Display walks from front to back and prints every item in the order they arrived

5.2 Key Code Snippets

```
void enqueue(T data) {

        Node<T>* newNode = new Node<T>(data) ;
        // Create new node

        // If queue is empty , front and back both point to new node
        if  back == nullptr) {
          (
            front = newNode
            back  = newNode;
                            ;
        } else
              {
            // Fix 75
            back->next = newNode ;
            // Link current back to new node
            back = newNode ;
            // New node becomes the new back
        }

        size++
        cout << "Enqueued successfully !" << endl;
    }
```

Display from “ menus.cpp ”

```
Node<Passenger>* current = pQueue.getFront() ;
int counter = 1
               ;
while (current != nullptr) {

    cout << "--- #" << counter << " (from front) ---" << endl;
    current->data.displayPassenger() ;
    current = current->next ;
    counter++
}           ;
```

```
void dequeue() {

        if  front == nullptr) {
          (
            cout << "Queue is empty , nothing to dequeue !!" << endl;
            return
        }         ;

        Node<T>* temp = front ;
        // Save front node
        front = front->next ;
        // Move front to next node

        // If queue becomes empty after dequeue , reset back to null
        if  front == nullptr) { back = nullptr ; }
          (
        delete temp
        size--     ;
        cout << "Dequeued successfully !" << endl;
    }
```

5.3 Screenshots

```
=========================================
          Booking Office Queue
=========================================
1) Display All (Front to Back)
2) Enqueue Office
3) Dequeue Office
4) Peek (View Front)
5) Find Office
0) Back to Main Menu
=========================================
Enter your choice : 2

################################
Please Enter the Office ID (10 digits) : 0000000001
################################

################################
Please Enter the Office Name: Airbnb
################################

########################################
Please Enter the Location of the Office : RUH
################################

Enqueued successfully !@
```

```
=========================================
          Booking Office Queue
=========================================
1) Display All (Front to Back)
2) Enqueue Office
3) Dequeue Office
4) Peek (View Front)
5) Find Office
0) Back to Main Menu
=========================================
Enter your choice : 1

--- #1 (from front) ---
=====================================
          Office Profile
=====================================
Office ID       : 0000000001
Office Name     : Airbnb
Office Location : RUH

--- #2 (from front) ---
=====================================
          Office Profile
=====================================
Office ID       : 0000000002
Office Name     : Booking.com
Office Location : DBX
```

```
=========================================
          Booking Office Queue
=========================================
1) Display All (Front to Back)
2) Enqueue Office
3) Dequeue Office
4) Peek (View Front)
5) Find Office
0) Back to Main Menu
=========================================
Enter your choice : 3

Dequeued successfully !
```

6. Challenges and Lessons Learned

Honestly the hardest part was the delete by position function in the Linked List. It kept giving wrong results and we could not figure out why at first. After sitting with it for a while we realized the problem was the traversal , we were stopping at the target node instead of the one before it , By the time you reach the target you have already lost the reference you need to rewire the list , We drew it out on paper step by step and once we saw it visually it made sense and the fix was straight forward

Another bug that took us by surprise was in the Stack destructor , The loop that was supposed to free all the nodes from memory had a logical error , the pointer was moving in the wrong way inside the loop so some nodes were being skipped and not deleted , It was not something that crashed the program immediately which made it harder to notice , Once we spotted it and fixed the pointer order everything worked cleanly

The input buffer issue was small but annoying , Whenever we read a number from the user and then immediately asked for a text input , the text input would just skip and read nothing , It turned out the newline character from pressing Enter after the number was still sitting in the buffer and the text reader picked it up as an empty input. Adding one line to clear the buffer after every number input fixed it completely , but we spent more time on it than we should have

Working as a team on one codebase was a challenge we did not fully expect , Everyone was writing their own file separately and when we put it all together things started breaking , wrong include paths , different assumptions about folder structure , small naming differences , We fixed it by agreeing on how the project should be organized early on and setting up a Makefile so everyone was building the same way , That experience alone taught us a lot about how real team projects actually work

Setting up the Makefile was something none of us had done before , We had all written CPP code but always compiled single files manually , When the project grew into multiple files across multiple folders we needed a way to build everything together , We got it wrong a few times , the include paths were off and the file order caused errors we did not understand at first , After fixing it the whole project compiled with one command and we realized how much time we had been wasting before

7. Team Member Contributions

Member Name	Tasks / Contribution	Percentage
Mohammed al-badiah	Project leader , managed the overall structure , wrote main.cpp , menus.cpp , menus.h , set up and configured the Makefile for the entire project , fixed bugs across multiple files including the Stack destructor and delete by position , improved the UI across all menus , wrote the README	30%
Othman al-thabit	Wrote Ticket.h and Flight.h , helped with UI fixes	17.5%
Rakan al-harbi	Wrote BookingOffice.h and Passenger.h , implemented all input validation and setter functions for both classes	17.5%
Abdullah Al-nooh	Wrote Stack.h and Queue.h , implemented push , pop , enqueue , dequeue , peek , and all related logic	17.5%
Ahmed Al-otaibi	Wrote LinkedList.h , implemented the Node structure , insert , delete , modify , find , and display operations	17.5%

8. Conclusion

We started this project with the goal of building an Airline Management System and ended up learning a lot more than we expected , The system works fully , all four entities can be managed through Linked Lists , Stacks , and Queues , and every required operation runs correctly , Using generic templates meant the data structure code was written once and reused for all four data types , which kept the codebase clean and saved a lot of repetition

Comparing the three structures side by side was one of the most valuable parts of the project , The Linked List gave us the most control , we could reach any position , modify anything , and delete from anywhere , The Stack was simpler but limited to the top , which makes sense when you only care about the most recent item , The Queue was the most natural fit for an airline scenario , it respects the order people arrive and processes them fairly , just like a real boarding line

We are finishing this project with a better understanding of data structures , stronger debugging skills , and a real sense of what it takes to deliver a complete software project as a team , Every bug we fixed and every problem we solved together made us more confident in our ability to build and manage a real CPP App from start to finish